

Greedy
Answer Template:
Proof: exchange argument

Author: Wenyu (Wendy) Fan

1. **Define.**
• Greedy Solution $G = (g_1, g_2, \dots, g_k)$, ordered by the greedy sorting criterion
• An Optimal Solution $OPT = \{o_1, o_2, \dots, o_m\}$, sorted by the **same** criterion
• We want to show **$k=m$** .

2. **Compare solutions.**
• Find the largest index r such that $g_1=o_1, g_2=o_2, \dots, g_r=o_r$ (i.e. the first r elements in OPT and G are the same but not the $(r+1)$ -th element)
• **Case 1:** $r=\min(k,m)$,

- If $k=m$, we are done
- If $k \neq m \rightarrow k < m$ and $k > m$ both yield contradiction.

• **Case 2:** $r < \min(k,m)$, the **first mismatch is at index $r+1$** , $g_{r+1} \neq o_{r+1}$.

3. **Exchange Pieces.**
• Argue that the greedy rule forces **$(\text{measure})(g_{r+1}) \leq (\text{measure})(o_{r+1})$** , (i.e. Finish time)
• Replacing o_{r+1} by g_{r+1} in OPT , $OPT' = \{o_1, o_2, \dots, o_r, g_{r+1}, o_{r+2}, \dots, o_m\}$.
• **Prove cost doesn't increase** (g_{r+1} does not overlap any of $\{o_1, \dots, o_r\}$ and $\{o_{r+2}, \dots, o_m\}$).
• OPT' still feasible and optimal. $|OPT'| = |OPT| = m$.

4. **Iterate Until Full Alignment.**
• Replace OPT with OPT' . Now the first $r+1$ elements in OPT and G are the same.
• If $r+1 = \min(k,m) \rightarrow$ **Case 1**
• Otherwise, we have a new mismatch at index $(r+1)+1$. Continue iteratively until the first $\min(k,m)$ elements match. In every swap, the solution stays feasible and keeps size m .

5. **Conclusion:**
• every step maintains feasibility and preserves the number of intervals,
• the " $r=\min(k,m)$ " case rules out $k < m$ and $k > m$, we deduce $k=m$.
• The greedy solution G has the same number of elements as any OPT , so G is optimal.

Applications List (definition, solution, time):
• **Interval Scheduling:** $O(n \log n)$
• Given a set of jobs, each with a start and finish time, select a **maximum-size subset of non-overlapping jobs**. $\rightarrow$ Sort jobs by earliest finish time, then scan through, picking each job that starts after the last one you selected.

• **Knapsack (0-1):**
• Given n items each with weight w_i and value b_i and capacity W , choose a subset to **maximize total value under weight $\leq W$** , but **take each item whole or not at all**; **pseudo-polynomial** (polynomial in size of numbers not their bit length) DP solution in $O(nW)$, polynomial in n and the numeric target W , but not polynomial in $\log W$.
• **Optimization ver.: NP-hard, Decision ver.: NP-Complete**
• **Fractional Knapsack:** $O(n \log n)$.
• **allows taking partial items(difference)**. "picking item with highest b_i/w_i " rule is provably optimal
• **Interval Partitioning:** $O(n \log n)$.
• Given n intervals, assign them to the **minimum number of "rooms"** so that no two overlapping intervals share a room $\rightarrow$ Sort intervals by start time, use a min-heap of end times, if the earliest-ending room is free (end $\leq$ start), reuse it; otherwise open a new room.

• **Scheduling to Minimize Lateness:** $O(n \log n)$
• Given n jobs each with processing time t_i and due date d_i , find an order that **minimizes the maximum lateness $L = \max_i (f_i - d_i)$** $\rightarrow$ Sort jobs in nondecreasing order of due date.

• **Shortest path:** Dijkstra, Binary-heap $\rightarrow O((n+m) \log n)$; Fibonacci-heap $\rightarrow O(m \log n)$
• **Minimum spanning trees:** Given a connected weighted graph, find a subset of edges of minimum total weight that connects all vertices. $\rightarrow$ Kruskal: sort edges and union-find, $O(m \log m) \approx O(m \log n)$. Prim's: priority-queue over vertices, $O(m + n \log n)$

Coping with hardness
(sacrifice 1 of: optimality, poly-time, arbitrary instances)
Solving NP-Hard Problems on restricted input instances:
Independent Set on Trees: Greedy, $O(n)$ in postorder.
Weighted IS on Trees: DP, $O(n)$ in postorder.
Solving restricted instances
Finding Small Vertex Covers: Let (u,v) be an edge of G . G has a vertex cover of size $\leq k$ **iff** at least one of $G \setminus \{u\}$ and $G \setminus \{v\}$ has a vertex cover of size $\leq k-1$. Can be solved in $O((2^k k)kn)$ time.

Approximating the optimal solution
• Sacrifice optimality, but still run in polynomial time.
• Approx ratio = Cost of our solution / Cost of optimal solution
• An approximation algorithm for a minimization problem comes with an approximation guarantee:

- Approximation ratio $\leq c$
- Cost of our solution $\leq c \cdot$ cost of optimal solution

• **Vertex Cover:** S is a **2-approximation**.
• **Weighted VC:** For any vertex cover S and any fair prices
• **Pricing method.** Set prices and find vertex cover simultaneously. **a 2-approximation**.

• **Load Balancing:** **makespan** $= L = \max L_i$
1. **List Scheduling:** σ is processing time, assign each job to machine with smallest load so far. $O(n \log n)$ using a priority queue.
2. **Longest processing time (LPT):** Sort n jobs in descending order of processing time, and then run list scheduling algorithm. **a 4/3-approximation**.

i&D
Answer Template: General Idea

0. **Preprocessing:** After pre-processing, we will use the recursive function X which takes input A and outputs R in A .

1. **Divide**
a. **Base cases:** trivial, can be solved in constant runtime, always reachable. $n = 1$ when there is only one element.
b. **Substances:** smaller, independent, the exact same problem. Divide A into left and right half as AL and AR .
2. **Delegate:** Recursively solve subinstances **independently**. Apply X on AL and AR and let L and R be the majority element returned by $\text{Major}(AL)$ and $\text{Major}(AR)$.
• **Combine:** Spot a general pattern from base case, apply to combining general substance solutions. **Recursive characterization of X contains 3 cases.** If... return... Else, extra computation, return...

Proof: Induction
1. **Base case:** $n=1$
2. **Induction hypothesis:** Assume that the algorithm is correct for all k , with $1 \leq k \leq n-1$.
3. **Inductive step:** Prove the algorithm is correct for $k=n$.
Time: The number of recursive calls $k(n)$. The size of the input passed to each recursive call relative to the current size of the input (n/d) . The amount of work performed at each recursive call relative to the size of the input $f(n)$

$$T(n) = \begin{cases} \alpha T(n/b) + f(n) & \text{for } n \geq d \\ c & \text{for } n < d \end{cases}$$

1. if $f(n) = O(n^{\log_b a - \epsilon})$ for $\epsilon > 0$ then $T(n) = O(n^{\log_b a})$,
2. if $f(n) = O(n^{\log_b a} \log^k n)$ for $k \geq 0$ then $T(n) = O(n^{\log_b a} \log^{k+1} n)$,
3. if $f(n) = O(n^{\log_b a + \epsilon})$ and $a f(n/b) \leq \delta f(n)$ for $\epsilon > 0$ and $\delta < 1$ then $T(n) = O(f(n))$,

Unrolling: $T(n) = 2^* T(n/2) + O(n) = c^* n + 2^* c^* (n/2) + 4^* c^* (n/4) + \dots$
• $T(n/(2^k k))$ reaches the base case when $n/(2^k k) = 1$,
• $\Rightarrow k = \log_2(n)$. Substituting $k = \log_2(n)$.
• $\Rightarrow n = 2^k T(1) + c^* n[1 + (1/2) + (1/4) + \dots + (1/n)]$

Useful Formula: $T(n) = 2T(n/2) + n \log n = O(n \log^2 n)$ (re-sort the combined n elements)

$T(n) = 2 T(n/2) + O(n)$	$T(n) = O(n \log n)$	(Merge sort)
$T(n) = 2 T(n/2) + O(\log n)$	$T(n) = O(n)$	
$T(n) = 2 T(n/2) + O(1)$	$T(n) = O(n)$	$T(n) = 4T(n/2) + O(1) = n^2 = O(n^2)$
$T(n) = T(n/2) + O(n)$	$T(n) = O(n)$	
$T(n) = T(n/2) + O(1)$	$T(n) = O(\log n)$	(Binary Search)
$T(n) = T(n-1) + O(n)$	$T(n) = O(n^2)$	
$T(n) = T(n-1) + O(1)$	$T(n) = O(n)$	

Applications List (general idea, time):
• **Maximum Contiguous Subarray:** $T(n) = 2 T(n/2) + O(n) \rightarrow T(n) = O(n \log n)$.
Given an array $A[]$ of n numbers, find the **maximum sum found in any contiguous "subarray"**.
1. **MCS in $A[1..n/2]$ & $A[n/2+1..n]$:** Recursively compute the maximum contiguous-subarray sum in the left & right half.
2. **MCS that spans across $A[n/2]$**
a. Scan left from index $n/2$ to find the largest suffix-sum ending at $A[n/2]$.
b. Scan right from index $n/2+1$ to find the largest prefix-sum starting at $A[n/2+1]$.
c. Add those two values to get the best "cross-middle" sum.
• **Counting Inversions** $T(n) = 2 T(n/2) + O(n) = O(n \log n)$.
1. **CI in $A[1..n/2]$ & $A[n/2+1..n]$:** Recursively count inversions in the left & right half and produce a sorted left & right subarray.
2. **CI that spans across $A[n/2]$:** Merge the two already-sorted halves into one sorted array, and during the merge: -
a. Whenever you take an element from the right half because it is smaller than the current left-half element, add the number of remaining items in the left half to your cross-inversion count.
b. The merged output is a fully sorted $A[1..n]$, and the total cross-inversions is exactly the number of left-vs-right inversions.
• **Closest-Pair:** Preprocessing: $O(n \log n)$. $T(n) = 2T(n/2) + O(n) = O(n \log n)$
• Given n points in the plane, find a pair with smallest Euclidean distance between them. Only need to consider points within δ of line L . Sort points in 2D-strip by their y coordinate. Only check distances of those within 7 positions in sorted list.
• **Majority Element:** $T(n) = 2T(n/2) + O(n) = O(n \log n)$
• If the array has a single element, we return that element as the majority element. Otherwise, we break the input array A into two halves, recursively call the algorithm on each half, and then test in A if either of the elements returned by the recursive calls is indeed a majority element of A . If that is the case we return the majority element; otherwise, we report that there is "no majority element".
• **Correctness:** If there is no majority element of A then the algorithm must return "no majority element" since no matter what the recursive call returns, we always check if an element is indeed a majority element. Otherwise, if there is a majority element x in A we are guaranteed that one of our recursive calls will identify x (by part a)) and the subsequent check will lead the algorithm to return x .
• **Largest Complete Subtree:** $\text{LCS}(\text{root}) = \min(\text{LCS}(L), \text{LCS}(R)) + 1$.

Base Exchange Rule: $\log_a(x) = (\log_b(x))/(\log_b(a))$
Product Rule: $\log_a(xy) = \log_a(x) + \log_a(y)$
Power Rule: $\log_a(x^b) = b \log_a(x)$

Graph
 $\sqrt{n} < n < n \log n < n^k < k^n < n! < n^n$ for
1. **Complete Graphs:** is one where every two vertices are connected with an edge.
• DFS Tree: a simple path; BFS Layers: $L_0 = \{s\}$, $L_1 = V \setminus \{s\}$
2. **Traversal Time Complexities (Adjacency-List):** BFS & DFS: $O(n + m)$
3. **Bridge (cut-edge):** an edge whose removal increases the number of connected components.
4. **Trees:** connected, acyclic undirected graphs.
• Every edge in a tree is a bridge. A tree on n vertices has exactly $n-1$ edges.
5. For every simple undirected graph, $\sum_{v \in V} \deg(v) = 2|E|$.

DP
Answer Template:
General Idea

1. **Define Subinstances** $OPT[i,j] =$
• Good subinstances should satisfy: ***overlapping**

- There is only **a polynomial number** of them
- The **original instance** is either a subinstance in itself or can easily be derived from one
- There exists a **natural ordering** between the subinstances (i.e., your dependency graph should be directed and acyclic)
- You should be able to compute the solution to a subinstance from some number of smaller subinstances using a well-defined recurrence

2. **Define Recurrence** $OPT[i,j] = \max\{, \}$
• Assume you include something in your optimal solution
• What does that say about the structure of the **remainder** of the solution?
• Formulate this **in terms of subinstances**.
3. **Define Base case** (satisfies the 3 following properties):
• Trivial to solve **without the need for smaller subinstances**.
• Can be solved **in constant time**
• In a **top-down** approach, ensure it will always be reached; In a **bottom-up** approach, ensure you will never require a smaller subinstance to build up your solution.

4. **Evaluate subinstances in the right order**
Ensure that when computing the solution to a subinstance, the subinstances it depends on have previously been solved.

Proof: Induction (0-1 knapsack)
1. **base cases are sufficient.** Show every path of dependencies hits a base case you've defined. The base cases are sufficient as each $OPT(i, w)$ depends on some $OPT(i-1, w)$. Thus, the recurrence will always end up at $OPT(0, w)$ for some w .
2. **base cases are correct.** Show each base case's value really satisfies the problem statement. We have $OPT(0, w) = 0$ since the most value that can be achieved with no items is 0.
3. **recurrence for $OPT(i)$ is correct.** (Inductive Hypothesis + step) Let $S(i, w)$ be the optimal solution for the first i items with weight limit w . The recurrence for $OPT(i, w)$ is correct because:
• if $w_i > w$, then $S(i, w)$ cannot contain item i and so is a subset of the first $i-1$ items with weight at most w , so $S(i, w) = S(i-1, w)$ and $OPT(i, w) = OPT(i-1, w)$
• if $w_i \leq w$, then either item i is in $S(i, w)$ or not: ...

- if item i is in $S(i, w)$, then the remaining items of $S(i, w)$ is a subset of the first $i-1$ items with weight at most $w - w_i$, so $S(i, w) = \{i\} \cup S(i-1, w - w_i)$ and so $OPT(i, w) = w_i + OPT(i-1, w - w_i)$.
- else, $S(i, w)$ is a subset of the first $i-1$ items with weight at most w , so $S(i, w) = S(i-1, w)$ and $OPT(i, w) = OPT(i-1, w)$.

• Finally, the optimal value for the original problem is $OPT(n, W)$ as by definition of OPT , this is the most value that can be achieved by a subset of all n items with a weight limit of W .

4. **Evaluation Order is correct.** Show that your loops or recursion really do compute the smaller subproblems before the larger ones. We fill the DP table in row-major order. Our recurrence only ever reads entries from row $i-1$. Since when the outer loop is at i , we have already completed row $i-1$, by the time we compute each $OPT(i, w)$, all needed $OPT(i-1, w)$ are available. Hence the evaluation order respects the dependency graph.

Time & Space 1. The number of subinstances you need to compute to calculate the solution to your original instance. 2. The time it takes to compute each subinstance.
• **Space:** 1. The space required is dominated by the number of subproblems $\times$ space per subproblem. We define $OPT[i, w]$ ($0 \leq i \leq n, 0 \leq w \leq W$) and store these in an $(n+1) \times (W+1) \times (n+1)$ array. 2. Total space: $O((n+1)(W+1)) = O(nW)$.
• **Time:** 1. Setting every entry in the $(n+1) \times (W+1)$ table to 0 takes $O((n+1)(W+1)) = O(nW)$. 2. Fill DP table. For each item $i=1..n$ and each capacity $w=0..W$, compute $OPT[i, w] = \max\{OPT[i-1, w], w_i + OPT[i-1, w - w_i]\}$ in $O(1)$ time (a couple of array lookups, one addition, one comparison). Total for this phase: $O(nW)$. 3. Reading off $OPT(n, W)$ (the final answer) takes $O(1)$. 4. Thus the total time is $O(nW) + O(nW) + O(1) = O(nW)$.

"tree-DP" e.g. Maximum-weight independent set on a tree
1. Root the tree. Pick any node as root (call it r), and direct all edges "away" from r . Now every node u has a (possibly empty) set of children $C(u)$.
2. Decide your states. Think about what choices you have at a node u , and what information you need to pass up to its parent.
• $dp[u][0]$ = best total weight in u 's subtree if we do not pick u .
• $dp[u][1]$ = best total weight in u 's subtree if we do pick u .
3. Write the recurrence. Express each DP value at u in terms of the DP values of its children $\{v \in C(u)\}$. That usually means summing over children, taking maxima or minima, maybe adding edge weights, etc.
• **Base:** u is a leaf. $dp[u][0]=0, dp[u][1]=w(u)$.
• **Recurrence:** Suppose $C(u) = \{v_1, v_2, \dots\}$.
$$\begin{cases} dp[u][0] = \sum_{v \in C(u)} \max(dp[v][0], dp[v][1]) \\ dp[u][1] = w(u) + \sum_{v \in C(u)} dp[v][0] \end{cases}$$

• **Root:** $\max(dp[r][0], dp[r][1])$.
4. Traverse in post-order. Do a DFS from the root. When you return from all of u 's children, you compute u 's DP values in $O(\deg(u))$. No cycles means you never revisit a node. $\sum \deg(u) = 2|E| = O(n)$.

Weight wgt	Value val	i \ c	0	1	2	3	4
wgt[i-1]	val[i-1]	0	0	0	0	0	0
1	5	1	0	5	5	5	5
2	11	2	0	5	11	16	0
constant k	15	3	0	0	0	0	0

wgt[i-1]	val[i-1]	0	0	0	0	0
1	5	1	0	5	10	15
2	11	2	0	5	11	16
3	15	3	0	0	11	0

Applications List (sub-instance, recurrence, base cases, time):
1D dp
• **Maximum-Sum Contiguous Subarray:** $O(n)$
• **Include or exclude $A[i]$? $\rightarrow$ pick max of two**
• $OPT(i) = \text{value of the optimal contiguous subarray ending at } i \text{ (possibly empty)}$
$$OPT(i) = \max_{1 \leq j \leq i} (A[j] + \dots + A[i]), 0).$$

• $OPT(i) = \begin{cases} \max(A[i], 0), & i = 1, \\ \max(OPT(i-1) + A[i], 0), & i > 1. \end{cases}$
• **Weighted Interval Scheduling:** $O(n \log n)$; if pre-sorted: $O(n)$
• **Include or exclude job? $\rightarrow$ two-way choice**
• $OPT(i) = \text{max-weight of a compatible subset chosen from jobs } 1, 2, \dots, i$.
• $OPT(j) = \text{max-weight of a compatible subset chosen from jobs } 1, 2, \dots, j$
• $OPT(j) = \begin{cases} 0, & j = 0, \\ \max(w_j + OPT(p(j)), OPT(j-1)), & j > 0. \end{cases}$
• **Longest Increasing Subsequence:** $O(n^2)$
• **Split at some $j < i$ that comes before i**
• $OPT(i) = \text{length of the longest increasing subsequence ending at } i$.
• $OPT(i) = 1 + \max_{1 \leq j < i} OPT(j)$,
$$OPT(1) = 1 \text{ (every single element by itself).}$$

• **Coin Change 1(Sample):** $O(nk)$, $k = \text{num of coin categories}$
• **derive it from smaller amounts by "try every coin"**
• $OPT(i) = \text{minimum number of coins needed to make amount } i$

$$OPT(i) = \begin{cases} 0, & n = 0, \\ -1, & n < 0, \\ \min_{\text{coin } c \in \text{coins}} \{dp(n - \text{coin}) + 1\}, & n > 0. \end{cases}$$

2D dp
• **0-1 Knapsack:** $O(nW)$
• **Include or exclude item i (change w or not)? $\rightarrow$ two-way choice**
• **iterate w downward from W to w_i**
• $OPT(i, w) = \text{max value of a subset of items } \{1, 2, \dots, i\} \text{ with weight limit } w \text{ for } 0 \leq i \leq n \text{ and } 0 \leq w \leq W$.
• $OPT[i, w] = \begin{cases} 0 & \text{if } i=0 \\ OPT[i-1, w] & \text{if } w_i > w \\ \max\{OPT[i-1, w], b_i + OPT[i-1, w-w_i]\} & \text{otherwise} \end{cases}$
• **Unbounded Knapsack** $O(nW)$
• **Include or exclude item i (change w or not)? $\rightarrow$ two-way choice**
• **iterate w upward from w_i to W**
• $OPT(i, w) = \text{max value using item types } 1, 2, \dots, i \text{ with weight limit } w$.

$$OPT(i, w) = \begin{cases} 0, & i = 0, \\ OPT(i-1, w), & w < w_i, \\ \max\{OPT(i-1, w), w_i + OPT(i, w - w_i)\}, & \text{otherwise.} \end{cases}$$

• **Longest Common Subsequence:** $O(nm)$
• **Compare $X[i]$ vs $Y[j] \rightarrow$ move $NW(\cdot)$ (match) or NW**
• $OPT(i, j) = \text{length of LCS}(X[1..i], Y[1..j])$
• Find LCS of any prefixes of X and Y (not just of same length)
$$OPT(i, j) = \begin{cases} 0 & OPT(i, 0) = 0 \text{ for all } i, OPT(0, j) = 0 \text{ for all } j \\ \max\{OPT(i-1, j-1) + 1, OPT(i-1, j), OPT(i, j-1)\} & \text{if } X[i] = Y[j] \\ \max\{OPT(i-1, j), OPT(i, j-1)\} & \text{if } X[i] \neq Y[j] \end{cases}$$

• **Shortest paths in graphs with negative edge weights** **Dijkstra's algorithm correctly computes the shortest path in a graph with non-negative edge weights & non-negative cycles.**
• Working space: $O(n)$ Total space (including input): $O(m+n)$
• Running time: $O(mn)$ worst case
• **Include/Exclude last edge**
• $OPT(i, v) = \text{length of shortest } v\text{-}t \text{ path } P \text{ using at most } i \text{ edges.}$
$$OPT(i, v) = \begin{cases} 0 & \text{if } i=0 \text{ and } v=t \\ \infty & \text{if } i=0 \text{ and } v \neq t \\ \min_{(v,w) \in E} \{OPT(i-1, v), \min_{(v,w) \in E} [OPT(i-1, w) + c_{v,w}]\} & \text{otherwise} \end{cases}$$

• **Coin Change 2:** In how many ways can you combine coins to sum to N ? (similar to unbounded knapsack) $O(nk)$, $k = \text{num of coin categories}$
• **Include or exclude coin i (change w or not)? $\rightarrow$ two-way choice**
• $OPT[i][v] = \text{number of ways using the first } i \text{ coins (coins}[1..i]) \text{ to make amount } v$.
$$OPT[i][v] = \begin{cases} 1, & v = 0 \text{ and } i = 0, \\ 0, & v \leq 0 \text{ and } i \neq 0, \\ OPT[i-1][v] + OPT[i][v - w_i], & \text{otherwise.} \end{cases}$$

dp over intervals
• **RNA secondary structure:** Time: $O(n^3)$ Space: $O(n^2)$
• **Split interval at some k paired with j**
• $OPT(i, j) = \text{max number of base-pairs in a secondary structure of the substring } [i..j]$
$$OPT(i, j) = \begin{cases} 0, & i \geq j - 4, \\ \max_{1 \leq k < j-4} [1 + OPT(i, k-1) + OPT(k+1, j-1)] \end{cases}$$

• **Segmented Least Squares:** $O(n^2)$
• **Choose where to start the last segment**
• $OPT(j) = \text{minimum cost for points } p_1, p_2, \dots, p_j$
• $e(i, j) = \text{minimum sum of squares for points } p_i, p_{i+1}, \dots, p_j$
• $C = \text{fixed penalty cost per segment.}$
$$OPT(j) = \begin{cases} 0, & j = 0, \\ \min_{1 \leq i < j} \{e(i, j) + C + OPT(i-1)\}, & j > 0. \end{cases}$$

Concepts: 1. Basic Definitions: A flow network on a directed graph has: • Two distinct nodes the source, s, and the sink, t. • Each edge has a capacity c(e), representing how much flow it can carry. • Flow f(e) through each edge. An edge is saturated if f(e)=c(e). A s-t flow also must satisfy two constraints: • Capacity: For each e∈E, 0 ≤ f(e) ≤ c(e) • Conservation: For each v∈V-{s,t}, ∑_{e in to v} f(e) = ∑_{e out of v} f(e) The value of a flow f is: v(f) = ∑_{e out of s} f(e) = ∑_{e into t} f(e) A s-t cut • is a partition (A, B) of V with s∈A and t∈B. • Capacity: cap(A,B) = ∑_{e out of A} c(e) 2. Flow-Cut Bounds & Duality • Max flow problem: in both P and in NP • Weak Duality (Max flows Min cut): V the flow f, V cut(A,B), v(f) ≤ capacity(A,B). • Corollary: if v(f)=cap(A,B), f is a max flow, (A,B) is a min cut. • Flow Value Lemma: v(f)=f_out(A)-f_in(A) • Max-Flow Min-Cut theorem: max v(f)=min cap(A,B). • Bounds for Any Cut: v(f) ≤ cap(A,B) (so "max flow value ≤ capacity of any cut"). 3. Residual Network & Augmenting Paths • Residual Graph G_f: • If c(e)-f(e)>0 → Forward edge (u,v) with capacity c(e)-f(e) • If f(e)>0 → Backward edge (v,u) of capacity f(e) • Augmenting Path: Any path from s to t in G_f. • f is maximum ⇔ there is no augmenting path in G_f. • If there is an augmenting path P, then we can push extra flow along that path. • b=bottleneck(P, f)=min capacity on edges of P in residual network. • Forward edges: increase f(e) by b. Backward edges: decrease f(e) by b. 4. Ford-Fulkerson & Variants (∞ of iterations, Cost per iteration, Time) • FF: ≤ F (value of max flow); O(m+n); O((m+n)F)=O(mnF) • Choose any augmenting path using DFS; • Edmonds-Karp #1: O(m log F); O(m log n) (greedy); O(m^2 log n · log F) • Choose maximum bottleneck augmenting path using priority-queue/Dijkstra • Capacity-Scaling: O(m log F); O(m); O(m^2 log F) • Choose 1/2-approximate max bottleneck path via restricting to edges of capacity ≥ Δ using DFS (Graph very dense) (Maximize throughput) • Edmonds-Karp #2: O(n m); O(m+n); O(n(m-m+n)) ≈ O(n m^2) • Choose shortest augmenting path using BFS (Need strict poly-time) 5. Verification & Complexity • Verify Max Flow in O(|V|+|E|): 1. Build residual graph G_f. 2. Run one BFS/DFS from s. 3. If you reach t ⇒ there is an augmenting path ⇒ not max. Otherwise ⇒ is max. • Integrality Theorem: If all capacities are integers, there exists a max flow f with all f(e) ∈ Z. 6. Circulations & Extensions • A circulation is a flow network but with supplies and demands. Each vertex v has a demand d(v). demand if d(v)>0, supply if d(v)<0, transshipment if d(v)=0. • A valid circulation also must satisfy two constraints: • Capacity: For each e∈E, 0 ≤ f(e) ≤ c(e) • Conservation: For each v∈V-{s,t}, ∑_{e in to v} f(e) - ∑_{e out of v} f(e) = d(v) Necessary condition: sum of supplies = sum of demands. Reduce to max flow: - ∑_{v: d(v)>0} d(v) = - ∑_{v: d(v)<0} d(v) ⇒ D • "hook up all the supply vertices to a super-s, demand vertices to a super-t" • For each v with d(v) < 0, add edge (s, v) with capacity -d(v); same for t. • Use flow networks to model: G has circulation iff G' has max flow value of D. Circulation with supply/demand and lower bounds • Capacity: For each e∈E, f(e) ≤ f(e) ≤ c(e) 7. Self-Check Questions • Matching vs Vertex-Cover (Undirected): |max matching| ≤ |min vertex cover|. • a max flow f and a min cut (A, B) with v(f)>0: f_in(A)=0, f_out(A)=0

Applications List (definition, maximum flow formulation, equivalence, time): Bipartite matching: • Input: undirected, bipartite graph G=(LUR,E) • Max matching: MGE is a matching if each node appears in at most one edge in M (matching constraint) • Equivalence: Max cardinality matching in G = value of max flow in G. ⇒: If there is a matching of size k, then there is a flow of value at least k. - Matching constraints ensure that we satisfy flow constraints - Size of matching ensure that flow value equals size of matching ⇐: If there is a flow of value k, then there is a matching of size at least k. - Flow integrality ensures we do not choose edges fractionally - Constraints of flow network (capacity, conservation) ensures we satisfy matching constraints (each vertex incident to at most one matching edge) - Flow Value Lemma shows that matching has same size as value of flow • Running Time: FF: O(mF)=O(mn); EK: O(m^2 log F)=O(m^2 log n). • Perfect Matching: A matching MGE is perfect if each node appears in exactly one edge in M. • Marriage Theorem: Let G=(LUR,E) be a bipartite graph with |L|=|R|. Then, G has a perfect matching iff |N(S)| ≥ |S| for all subsets S ⊆ L. Edge disjoint paths - Disjoint path problem: Given a digraph G = (V, E) and two nodes s and t, find the max number of edge-disjoint s-t paths. • Definition: Two paths are edge-disjoint if they have no edge in common. • Equivalence: Max number edge-disjoint s-t paths = max flow value. Network Connectivity (Min-Cut): • Definition: Given a digraph G = (V, E) and two nodes s and t, find min number of edges whose removal disconnects t from s. • Theorem: The max number of edge-disjoint s-t paths is equal to the min number of edges whose removal disconnects t from s. Baseball Elimination: • Flow from game node (x-y) to team node (x) equals # of matches between teams x and y that team x wins. • Flow from a team node (x) to the sink ensures no other team wins too many games. (w3+r3-w4 is exactly "Team 3's best-possible final wins" (w3+r3) minus "Team 4's current wins" (w4)) • Team i is not eliminated iff There is a flow in G of value at least the total number of remaining games between all pairs of teams. Image Segmentation; Survey Design

Network Flow Answer Template: General Idea: 1. Components Identification Instance of your problem • Input: Objects: Computers and base stations • Output: YES if...; Otherwise NO • Constraints 1. A computer can only be assigned to a single base station 2. A computer can only be assigned to a base station if it is within its radius 2. Properties Translation using components (reduction/network construction) Construct a flow network G=(V, E) with capacity function c as follows: • Add a source vertex s and a sink vertex t to V. • For each computer p∈P, add a vertex u_p to V and an edge (s, u_p) to E, and set c(s, u_p)=1. • For each base station b∈B, add a vertex v_b to V and an edge (v_b, t) to E, and set c(v_b, t)=L. • For each computer-station pair (p, b) ∈ P × B, if ||x_p - x_b|| ≤ r_p, then add an edge (u_p, v_b) to E, and set c(u_p, v_b) = ∞. Finally, set the target flow value to K = |P|. If its value equals, then all computers can be assigned (output YES); otherwise output NO. 3. State which algorithm you use to compute the max flow. Use Ford-Fulkerson to compute the maximum flow K in G. • If the resulting max-flow K is not equal to |P|, then no feasible assignment exists. 4. Solution Translation • Feasibility: If max-flow = |P|, every u_p sends exactly 1 unit to some v_b. • Reconstruction: For each computer p, find the station b such that the edge u_p → v_b carries 1 unit of flow. Assign p to b. Proof ⇒ If there exists a feasible assignment in our original problem instance then there must exist a max flow of value n in our reduced max flow instance (e.g. a base station that is within range for every computer). 1. Assume assignment and define a flow instance. • Assume there is a valid assignment g: P → B with each station's load ≤ L. • Define a flow f on the max flow instance as follows: f(u_p, v_b) = { 1, g(p) = b, 0, otherwise. a. For each computer p ∈ P, set c(s, u_p) = 1. b. For each computer-station pair (p, b) ∈ P × B with an edge (u_p, v_b), set f(u_p, v_b) = 1. c. For each base station b ∈ B, set f(v_b, t) = f_in(v_b). • By inspection of the reduction, the above steps assign a flow value to each edge in G. So f saturates the source cut, so v(f)=c(s), V(s)=|P|=K, v(f) = ∑_{p∈P} f(s, u_p) = |P| = K. 2. Check feasibility (A. capacity; B. integrality; C. conservation) • To establish that f is a valid flow, we need to check the flow constraints. a. f(e) ≤ c(e) for every edge e ∈ E: • (s, u_p) all assigned flow equal to their capacity. • (u_p, v_b) all have capacity ∞. • (v_b, t): capacity L, f_in(v_b) is constituted of 1 from (u_p, v_b) s.t. g(p)=b; The amount of p, by assumption, can't be greater than L. ⇒ f(v_b, t) = f_in(v_b) = # of g(p) (p) = b ≤ L = c(v_b, t). b. f_in(u) = f_out(u) for every u ∈ V - {s, t}: • At u_p: f_in(u_p)=1; (u_p, v_g(p)) is guaranteed to exist in G; f(u_p, v_b)=1 ⇒ f_out(u_p)=1 • At v_b: in-flow from computers equals out-flow to t by definition. c. Integrality (and non-negativity): all f(e) are integers ≥ 0. 3. Conclusion • Hence all flow-constraints are met, so f is a valid integral flow of value K. ⇐ If there exists a max flow of value n in our reduced max flow instance then there must exist a feasible assignment in our original problem (e.g. it must satisfy all edge capacity and flow conservation requirements) 1. Assume the flow (Max-flow existence). • Assume the flow network admits an integral flow f of value K = |P|. • By construction, K equals the capacity of the cut c({s}, V(s)), so the max flow is |P|. 2. Decoding to a solution a. Flow integrality. K is the capacity of the source cut, and by flow integrality theorem, we can assume that all flow values assigned to edges by f are integers. b. One unit per computer. Look at a computer-node u_p. The only incoming edge to u_p is (s → u_p) of capacity 1, and the only outgoing edges are (u_p → v_b). • Since the cut c({s}, V(s)) is saturated, every (s → u_p) must carry exactly 1 unit. • From the flow conservation at u_p, we can deduce that f_out(u_p)=f_in(u_p)=1 for each p∈P. c. Define the computer-station assignment g: P → B • f_out(u_p)=1 ⇒ u_p sends 1 unit in total • Flow integrality ⇒ each outgoing (u_p → v_b) has integer flow. • ⇒ exactly one edge (u_p, v_b) has a flow value of 1. • f(u_p, v_b)=1 ⇒ we set g(p)=b. • we only added (u_p, v_b) edges when ||x_p - x_b|| ≤ r_p ⇒ guarantees ||x_p - x_b|| ≤ r_p. • so the assignment is valid. d. Respecting station capacities. • Now consider a station-node v_b. f_in(v_b) is exactly the number of computers p with g(p)=b. (Each computer p with g(p)=b has f(u_p, v_b)=1) • Flow conservation ⇒ f_in(v_b)=f_out(v_b) • Capacity constraint ⇒ f_out(v_b) ≤ c(v_b, t)=L • Hence the max load assigned by g to any station is at most L. 3. Conclusion • We have produced a function g: P → B such that • Every computer p goes to some g(p) within its radius • No station b receives more than L computers • That is exactly a feasible assignment. Time 1. Flow network construction: Let |B|=m and |P|=n. Building an adjacency-list G takes Θ(mn) time since: - Adding source & sink: O(1); - For each p∈P, add u_p and edge (s, u_p): Θ(n); - For each b∈B, add v_b and edge (v_b, t) of capacity L: Θ(m); - For each (p,b)∈P×B, add (u_p, v_b): Θ(mn) Vertices: |P|+|B|+2 = n+m+2 ∈ Θ(m+n); Edges: |P|×|B|+n+m ∈ Θ(mn); Θ(mn) dominates. 2. Max-flow algorithm: Since max flow = C=c(s), V(s)=|P|=n, the FF using any reasonable augmenting path selection strategy would solve the max flow instance within O(|E|·C) = O(mn·n) = O(mn^2)

NP-completeness (Karp reduction from MNC) Answer Template: Show that problem B is NP, e.g. Partition Certificate: Two subsets of S: S1 and S2. Certifier: Check that S1US2=S. Check that the sum of S1 is equal to the sum of S2. Return YES if equal, otherwise NO. Correctness: • Certifier YES implies certificate is valid: All the requirements of the question are satisfied, therefore S1 and S2 must have equal sum and S1US2=S, therefore the certificate must be valid. • Certificate is valid implies certifier YES: If the certificate is valid then S1 and S2 must have equal sum and S1US2=S, covering exactly all element in S. This satisfies all conditions of the certifier and therefore it will output YES. Time: Finding the sum of the two subsets take O(|S|), check if they are equal takes O(1), check if their union equals S takes O(1). Therefore, the certifier runs in polynomial time. Show that problem B is in NP-Hard (is NP-complete) Reduction: A ⇒ B e.g. subset sum to partition. 1. Components Identification • Input: the decision-problem instance • Properties of YES instance: - ... • Exist (Certificate) s.t. - (Certifier) 2. Properties Translation using components Given an instance of SUBSET-SUM V=(v1,v2,...,vn) with target value t, let the sum of V be a. Consider the case that t = a/2. We construct a PARTITION-SUM instance S=(v1,v2,..., vn, a-2t). Correctness • YES for subset sum ⇒ YES for partition: Assume V is a YES SUBSET-SUM instance, then there must exist some subset V' that sum to exactly t, where the sum of the remaining integers is a-t. Therefore, there exists a partition of S into sets S1=V∪(a-2t) and sets S2=V'. Both of these sets sum to a-t. • YES for partition ⇒ YES for subset sum: Assume V is a YES PARTITION-SUM instance, then there must exist a partition of S into sets S1 and S2 whose sum are equal. Since the sum of S is 2a-2t, each of S1 and S2 sums to a-t. Assume S1 is the set that contains a-2t. Therefore, the remaining integers in S1 belong to V and sum to a-(a-2t)=t. To handle the case when t > a/2, note that there exists a subset that sums to t iff there exists a subset to a-t. Thus in this case, the Partition instance we create consists of the integers S=(v1,v2,..., vn, a-2(a-t)). The rest of the argument is similar to the above. Therefore, we have shown a Karp reduction from Positive Subset-Sum to Partition such that a Subset-Sum instance is a "yes instance" iff its corresponding Partition instance is also a "yes instance". Time: Calculate a takes O(n). Check if t > a/2 takes O(1). Copy the integers V and add a new integer takes O(n)+O(1). Therefore, the reduction is bounded by polynomial time. Since A is NP-hard, we conclude that B is also NP-hard. Concepts: Difference between a Cook reduction and a Karp reduction: • Karp (many-one) computes a single instance • Cook (Turing) makes multiple queries to an oracle for B. • Every Karp is a special case of a Cook with just one nonadaptive query. Three types of standard • Special case to general case: VERTEX-COVER ≤_p SET-COVER reductions: - Encoding with gadgets: 3-SAT ≤_p INDEPENDENT-SET. Decision ≤_p Search ≤_p Optimization 3-SAT ≤_p CLIQUE • D: Does there exist a matching of size>=k? Yes/No. • Binary Search 1-m (O(log m) calls to D) • O: size of optimum k = YES above? Number. • Remove node by node (O(m) calls to O) • S: Find a matching of size>=k. Set of edges. • O(1) call to S Transitivity: If X ≤_p Y and Y ≤_p Z, then X ≤_p Z. Equivalence: If X ≤_p Y and Y ≤_p X, then X ≡_p Y. Classes (P, NP, NP-hard, NPC) P: Decision problems for which there is a poly-time algorithm. NP: Decision problems X which have a poly-time certifier V s.t. For every YES-Instance x, there exists a certificate c such that V(s,c) = YES EXP: exponential-time; NP ≤ EXP. NP-hard: A decision problem such that every problem in NP reduces to it. NP-complete: A problem in NP such that every problem in NP polynomially reduces to it. Induction for DP base Case For amount 0 we wrote down 0 coins. That is obviously optimal, because you cannot use a positive number of coins and still sum to zero cents. Inductive Hypothesis Assume that for every amount strictly smaller than some number K (that is, for 0 ≤ k < K) the table entry dp[k] equals the true minimum number of coins. Inductive Step Consider amount K. • Any legal way of making K must end with one last coin of value 1, 2, or 5. Removing that coin leaves K-1, K-2, or K-5 cents respectively. • By the inductive hypothesis, dp[K-1], dp[K-2], and dp[K-5] (when those indices are non-negative) already store optimal answers for those smaller amounts. • Our algorithm tries each of those possibilities and picks the smallest resulting total. Call the answer it chooses A. • Suppose, for contradiction, that A is not optimal. Then there exists some other arrangement that uses fewer coins. Remove its last coin; the remainder is one of the smaller amounts mentioned above, and the number of coins it used for that remainder must be strictly less than the value we stored for that remainder in dp. That violates the inductive hypothesis. Therefore the contradiction shows A must in fact be optimal. By induction, dp[k] is optimal for every K from 0 up to n, so the value the algorithm returns, dp[n], is the true minimum number of coins. Running Time The loop touches each amount once and performs a constant amount of work each time, so the total running time is proportional to n; we say it runs in O(n) time. Ford-Fulkerson Algorithm - Initialize f to f(e) = 0 for every edge e - Compute residual network G_f - While there exists an s-t path P in G_f • f = Augment(P, f) • Update G_f - Return f Iterations. Path in G_f can be found in O(m+n) time using BFS. Augment(P) takes O(n) time. Updating G_f takes O(m+n) time.

Application List (definition, input, certificate, certifier=ε, Proper): Constraint Satisfaction problem: SAT; 3-SAT: Given CNF formula φ, where each clause has exactly 3 literals, does it have a satisfying truth assignment? e.g. φ=(x∨y∨z)∧(¬x∨y∨z). A propositional formula φ over variables X={x1,...,xn}. A truth assignment A: X → {T, F}, e.g. x=F, y=T, z=F. Check A(x) ∈ {T, F}. Check each clause in φ has at least one true literal. Circuit-SAT: Given a combinational circuit built out of AND, OR, and NOT gates, is there a way to set the circuit inputs so that the output is 1? Packing Problem: Independent Set: Given a graph G = (V, E) and integer k, is there a subset of vertices S ⊆ V such that |S| ≤ k, and for each edge at most one of its endpoints is in S? e.g. 1-2-3-4-1, k=2. G=(V,E), integer k. A subset S ⊆ V with |S| ≤ k. Check |S| ≤ k. Check ∀u, v ∈ S: (u, v) not in E. Clique: (A clique is a complete subgraph) Given a graph G=(V,E) and an integer k, does G=(V,E) contain a clique of size k? G=(V,E), integer k. A subset C ⊆ V with |C| ≤ k. Check |C| ≤ k. Check ∀u, v ∈ C and u ≠ v: (u, v) ∈ E. Steiner Tree: Given a graph, a subset of vertices X in that graph, an integer k, does there exist a subset S ⊆ V of size at most k such that G[XUS] is connected? e.g. V={1,2,3,4}, E={(1,4),(2,4),(3,4)}, x={1,2,3}, k=1 ⇒ S={4}. S ⊆ V with |S| ≤ k. Feedback Set: Given a directed graph G = (V, E) a feedback set is a set X ⊆ V with the property that G-X is acyclic. Given G and k, does G have a feedback set of size at most k? Graph G=(V,E), integer k. A subset S ⊆ V with |S| ≤ k. Remove S and check the remaining graph has no cycle. Covering Problem: Set-Cover: Given a set U of elements, a collection S1, S2, ..., Sm of subsets of U, and an integer k, does there exist a collection of k of these sets whose union is equal to U? U={F={S1,..., Sm}, integer k. C ⊆ {1,..., m} with |C| ≤ k. Check |C| ≤ k. Check ∪_{i=1}^k S_{i_i} = U. Vertex Cover: Given a graph G=(V, E) and integer k, is there a set of at most k vertices {t} that touches every edge? G=(V, E), C=a permutation(v1,...,vn) of the n nodes. Check the permutation contains each node in V exactly once (v_i, v_{i+1}) ∈ E for all i). Check there is an edge between each pair of adjacent nodes in the permutation (v_{n+1}) ∈ E. Sequencing Problem: 3-SAT-SDIR HAM C=HAM C=TSIP Ham-cycle: Given an undirected graph G = (V, E), does there exist a simple cycle C that visits every node? G=(V,E). A sequence of vertices (v1,v2,...,v|V|) listing each vertex exactly once. Check the sequence has length |V| and no repeats. Check for each i=1, ..., |V|-1, (v_i, v_{i+1}) ∈ E. Check (v|V|, v1) ∈ E. Directed Ham-Cycle: ... a simple directed cycle C... Ham Path: Same as Ham Cycle but a path instead of a cycle. Check the sequence has length |V| and no repeats. Check for each i=1,...,|V|-1, (v_i, v_{i+1}) ∈ E. Directed Ham Path: but a path instead of a cycle. Longest Path: Given a digraph G = (V, E), does there exist a simple path of length at least k edges? G=(V,E), integer k. A simple path (v1,...,v_{k+1}). Check each (v_i, v_{i+1}) ∈ E. Check all v_i distinct. TSP: Given a set of n cities and a pairwise distance function d(u, v), is there a tour of length ≤ D? Complete graph K_n with weight function w: E → Z^+, bound B. A permutation (v1, ..., vn). Sum w(v_i, v_{i+1}) plus w(v_n, v1) and check ≤ B. Degree-2 Spanning Tree (A Spanning Tree): For some fixed Δ=2, given a graph G = (V, E), is there a spanning tree of G with degree at most Δ? Graph G=(V,E). A set of edges T ⊆ E. Check T is acyclic; connected; |T|=|V|-1 and T ⊆ E; for each v ∈ V, deg_T(v) ≤ 2. Note that Δ is a fixed parameter, not part of the input (i.e., considered to be a constant, does not depend on n in what we saw). Partitioning Problem: 3-Coloring: Given an undirected graph G=(V, E) with n vertices, does there exist a proper coloring using at most 3 colors? G=(V, E). Coloring f: V → {1, 2, 3}. Check ∀v ∈ V: f(v) ∈ {1, 2, 3}. Check ∀(u, v) ∈ E: (u,v) = f(u) ≠ f(v). 4-Coloring Numerical problem: Subset Sum: Given a set of integers S and an integer k, is there a subset of S which sums to k? e.g. S={3,6,7,2,4,3}, t=10; a set S of integers, target T ∈ S. Check ∑_{x ∈ S'} x = k. Partition: Given a multiset S of non-negative integers, is there a partition of S into two subsets S1 and S2 such that the sum of S1 equals the sum of S2? (P2 LP3) Knapsack: Given a set of n items each with weight w_i and value v_i, a total weight limit W, and a minimum total benefit V, W=∑ w_i ⇒ i=1,2,3. Items ((w_i,v_i)), capacity W, target value V. Subset I ⊆ {1,...,n}. Check total weight ≤ W and total value ≥ V. Problem A is NP-complete for some fixed k, show it remains NP-complete for k' > k: 1. Take any instance of A (with k). 2. Add a small "gadget" that forces one extra unit of the resource (color, literal, subset, degree) to be used exclusively by that gadget. 3. (⇒) If the original instance is a YES-instance, you can satisfy the extended one by using the extra resource only for the gadget and solving the rest as before. (⇐) If the extended instance is solvable, then since the gadget consumed exactly one unit of the extra resource, the remainder is forced to satisfy the original problem with only k resources (can't rely on the extra), so the original instance was a YES-instance. 3 SAT ⇒ 4 SAT: Introduce a fresh Boolean variable a. Take each clause (x ∨ y ∨ z) and turn it into (x ∨ y ∨ z ∨ a) ∨ (x ∨ y ∨ z ∨ ¬a). 3Coloring ⇒ 4Coloring: Add one new vertex v_0 and connect it to every vertex of the original graph. In a 4-coloring, v_0 "isolates" one color (it must be distinct from all neighbors), so the neighbors can only use the other three. 2 Partition ⇒ 3 Partition: Add one new element of value equal to the target sum. 22 ⇒ DN spanning tree: for each original vertex u, attach (N-2) new leaf-neighbors. No perfect matching S = {2, 4, 5}. NS(S) = {2', 5'}. Yes: 1 0 1 hard-coded inputs inputs k=2 U = {1, 2, 3, 4, 5, 6, 7} k=2 S_1 = {3, 7} S_2 = {3, 4, 5, 6} S_2' = {1} S_3 = {2, 4} S_4 = {5} S_2' = {1, 2, 6, 7}